EPISODE 1 COMPANION FIELD GUIDE

101 Technical Alignment Methods

An introductory map for undergraduate computer science: what each method does, where it fits, and what can still go wrong.

Audience Introductory undergraduate CS
Format Ten method families + selection guide + study plan
Version October 2026

101

The list is a pipeline, not a leaderboard

Technical alignment is the work of making an AI system reliably pursue intended goals and constraints under ordinary use, uncertainty, distribution shift, and adversarial pressure. The 101 entries in this guide mix algorithms, data practices, evaluation protocols, runtime controls, and governance mechanisms. That is deliberate: deployed alignment is a system property.

10

families cover the lifecycle

Data, demonstrations, preferences, principles, oversight, interpretability, evaluation, runtime control, robustness, and multi-agent systems.

3

questions organize every method

Signal: what tells the system what is wanted? **Control:** where can behavior be changed? **Evidence:** how do we know the change worked?

Specify	Train	Inspect	Attack	Govern
values, policy, tasks	data, demos, feedback	features, circuits, probes	evals, red teams, shift	limits, recourse, updates

Four cautions for beginners

- **Behavior is not intent.** Passing a benchmark does not prove a model has the right internal objective.
- **A proxy is not the goal.** Rewards, rubrics, constitutions, and classifiers all simplify what people care about.
- **Alignment is plural.** People disagree about values, risk, and acceptable trade-offs.
- **Controls compose imperfectly.** More layers can help, but interfaces create new failure modes.

What “technical” means here

The field includes optimization and neural-network analysis, but also corpus engineering, human-computer interaction, security testing, mechanism design, and institutional release processes. A method belongs when it materially changes the probability or consequences of misaligned behavior.

> **Best mental model:** build a safety case. Each technique contributes one piece of evidence or one control; none is a certificate by itself.

Alignment controls sit at different points in the system

1 · Data prior	Filter, balance, decontaminate, and audit what pretraining teaches.	Methods 1-10
2 · Demonstrated behavior	Show the model examples of instructions, dialogue, tools, and safe conduct.	11-19
3 · Preference signal	Translate rankings, feedback, process labels, or verifiable outcomes into updates.	20-33
4 · Explicit principles	Write, ground, test, and publish rules the model should apply.	34-43
5 · Oversight channel	Amplify human judgment when direct evaluation is too difficult.	44-53
6 · Internal evidence	Probe and intervene on representations, features, pathways, and circuits.	54-65
7 · Behavioral evidence	Benchmark expected failures and actively search for unknown ones.	66-77
8 · Runtime boundary	Wrap the model with hierarchy, classifiers, moderation, and abstention.	78-87
9 · Robustness layer	Adversarially train, unlearn, or reroute risky behaviors.	88-95
10 · Ecosystem design	Shape correction, incentives, coordination, participation, and provenance.	96-101

Important: the episode's “101 methods” are not all equally mature or independent. Some are close variants, some are evaluation frameworks, and some are sociotechnical controls. The value of the list is coverage, not taxonomy purity.

If you can change...	Start with...	Then add...
Only prompts and orchestration	78-87 runtime techniques	66-77 evaluations and red teams
Fine-tuning weights	11-19 demonstrations; 22-27 preference methods	34-43 principles; 88 robustness
The entire training run	1-10 data controls	20-33 feedback; 54-65 interpretability
Agent permissions and institutions	77 release gates; 96-101 ecosystem methods	Long-horizon, multi-agent evaluations

Pretraining and data curation

Methods 1-10

Shape the base distribution before the model learns instructions. [11]

1 Quality filtering with classifiers

Score and retain higher-value documents before pretraining.

Watch: Classifiers encode cultural and domain bias; audit false removals.

2 Deduplication

Remove exact and near-duplicate text to reduce memorization and skew.

Watch: Dedup can erase legitimate repetition and minority-language material.

3 Personal-data scrubbing

Detect and redact personally identifying information before training.

Watch: Pattern matching misses context and can over-redact benign text.

4 Toxicity filtering

Downweight or remove highly harmful content from the training mix.

Watch: Over-filtering can impair recognition, research, and reclamation contexts.

5 Data mixing and domain reweighting

Optimize the proportions of code, books, web, and specialist domains.

Watch: A better average can still under-serve a low-resource domain.

6 Benchmark decontamination

Find and remove evaluation items or close paraphrases from training data.

Watch: Contamination detection is incomplete; report the procedure, not certainty.

7 Rights and opt-out filtering

Exclude data based on licensing, consent, or documented opt-out rules.

Watch: Legal compliance and normative legitimacy are related but not identical.

8 Synthetic textbook-style data

Generate curated explanations and examples to improve data quality.

Watch: Synthetic corpora can amplify teacher errors and reduce diversity.

9 Multilingual balancing

Adjust language proportions and quality thresholds across languages.

Watch: Token balance does not guarantee cultural competence or equal quality.

10 Corpus bias auditing

Measure representation, associations, and missing populations before training.

Watch: An audit diagnoses a corpus; it does not by itself correct behavior.

Supervised fine-tuning

Methods 11-19

Teach instruction following by imitation of selected demonstrations. [1]

11 Instruction tuning

Fine-tune on many tasks expressed as natural-language instructions.

Watch: Generalization depends on task diversity and demonstration quality.

12 Human demonstration tuning

Train on expert-written examples of desired behavior.

Watch: Demonstrators may disagree; document rubrics and adjudication.

13 Reasoning-trace distillation

Transfer worked reasoning from a stronger teacher to a student model.

Watch: Visible rationales may be unfaithful and can leak sensitive details.

14 STaR

Generate rationales, keep successful ones, and iteratively fine-tune.

Watch: Verification of final answers may reward brittle or spurious reasoning.

15 Rejection-sampling fine-tuning

Sample many completions, select high-scoring ones, and train on winners.

Watch: The selector becomes a proxy objective and can favor style over truth.

16 Teacher-model distillation

Use a stronger model to produce training examples for a smaller model.

Watch: The student inherits the teacher's blind spots and policy choices.

17 Multi-turn dialogue tuning

Train on conversations that require context tracking across turns.

Watch: Long-context success in training does not ensure deployment persistence.

18 Tool-use and function-call tuning

Teach structured calls to calculators, search, code, and external APIs.

Watch: Correct syntax is not safe authority; permissions and validation remain essential.

19 Safety demonstration tuning

Train examples of safe assistance, bounded refusal, and escalation.

Watch: Refusal patterns can overgeneralize and suppress harmless help.

Human feedback and preference optimization

Methods 20-33

Turn rankings, binary feedback, or verifiable outcomes into parameter updates. [1][2]

20 RLHF with PPO

Fit a reward model from rankings, then optimize a policy with reinforcement learning.

Watch: Reward hacking, labeler bias, and training instability require monitoring.

21 Bradley-Terry reward modeling

Convert pairwise preferences into latent reward differences.

Watch: The model assumes a simple preference structure that may not fit disagreement.

22 Direct Preference Optimization (DPO)

Optimize preferred over rejected responses without an explicit reward model.

Watch: Still depends on preference-pair quality and a reference policy.

23 Identity Preference Optimization (IPO)

Use a conservative preference objective designed to limit overfitting.

Watch: A different loss does not remove noisy or contradictory labels.

24 Kahneman-Tversky Optimization (KTO)

Learn from desirable or undesirable examples without paired comparisons.

Watch: Binary feedback can hide why an answer was judged acceptable.

25 Odds Ratio Preference Optimization (ORPO)

Combine supervised learning and preference shaping in one objective.

Watch: Joint optimization simplifies the pipeline but couples two failure modes.

26 Simple Preference Optimization (SimPO)

Use a reference-free, length-normalized preference objective.

Watch: Length control does not solve factual or normative misspecification.

Human feedback and preference optimization

Methods 20-33

Turn rankings, binary feedback, or verifiable outcomes into parameter updates. [1][2]

27 Contrastive Preference Optimization (CPO)

Anchor preference learning with a supervised contrastive term.

Watch: Anchors can preserve desirable skill or freeze undesirable bias.

28 Reinforcement learning from AI feedback

Use model-generated judgments to scale preference labeling.

Watch: Judge-model bias and correlated error can propagate at scale.

29 Best-of-N selection

Generate several candidates and return the highest-scoring one.

Watch: Selection-time gains can intensify reward-model exploitation.

30 Process supervision

Score intermediate reasoning steps rather than only final outcomes.

Watch: Step labels are expensive and visible reasoning may be strategically incomplete.

31 Outcome supervision

Reward whether the final answer passes a task-specific check.

Watch: Correct outcomes can conceal shortcuts, luck, or unsafe intermediate actions.

32 Group Relative Policy Optimization (GRPO)

Compare groups of sampled answers to estimate relative advantages without a critic.

Watch: Requires reliable rewards; group normalization can be sensitive to sampling.

33 Online or iterative DPO

Generate fresh on-policy preference data across repeated tuning rounds.

Watch: Feedback loops can narrow diversity or amplify judge errors.

Constitutional and principle-driven approaches

Methods 34-43

Make behavioral rules explicit enough to critique, train, and audit. [3][14]

34 Constitutional AI

Use written principles for critique, revision, and AI-generated preference feedback.

Watch: The constitution's authorship and conflict rules remain governance choices.

35 AI self-critique and revision

Ask a model to identify defects in a draft and produce a corrected answer.

Watch: A model may repeat its own blind spots with greater confidence.

36 Deliberative alignment

Train models to reason over a safety specification before answering.

Watch: Reasoning traces are not guaranteed to reveal the true causal process.

37 Collective Constitutional AI

Elicit principles through public participation and synthesize them into a constitution.

Watch: Recruitment, aggregation, and minority protections determine legitimacy.

38 SALMON

Bootstrap principle-following reward signals from model-generated principles and critiques.

Watch: Self-generated supervision needs external anchors against drift.

39 In-context principle following

Place principles and examples in the prompt without changing model weights.

Watch: Fast and reversible, but vulnerable to context conflict and injection.

40 Grounded Constitutional AI

Derive principles from human reasons and value statements, not preference labels alone.

Watch: Reason sampling and synthesis can still erase dissent.

41 C3AI constitution engineering

Systematically craft, test, and revise constitutions as measured artifacts.

Watch: Benchmark coverage limits what the process can validate.

42 Inverse Constitutional AI

Infer candidate principles that explain observed human preferences.

Watch: Many incompatible principles can fit the same behavioral data.

43 Behavioral specification training

Train against a published model-behavior specification and hierarchy.

Watch: Public rules improve transparency but do not prove faithful implementation.

Scalable oversight

Methods 44-53

Help humans evaluate work that may be too difficult to judge directly. [4][5]

44 AI safety via debate

Two systems argue opposing positions for a human judge.

Watch: Persuasion skill, judge bias, and collusion can defeat truth-seeking.

45 Iterated distillation and amplification

Amplify a human with many assistants, then distill the result into a stronger model.

Watch: Errors can compound across iterations; decomposition quality is central.

46 Recursive reward modeling

Use AI-assisted evaluation to train reward models for progressively harder tasks.

Watch: Each recursion can inherit and magnify earlier evaluator defects.

47 Weak-to-strong generalization

Study whether a weaker supervisor can elicit capability from a stronger model.

Watch: Current experiments are proxies, not proof for superhuman oversight.

48 Sandwiching evaluations

Compare weak humans plus AI against a stronger human reference group.

Watch: The stronger group may itself be wrong or incomplete.

49 Self-critiquing models

Train systems to surface flaws that humans can verify more cheaply.

Watch: Critiques may omit strategic or hard-to-detect failures.

50 Prediction-market oversight

Aggregate agents' beliefs through bets or scoring rules.

Watch: Markets need verifiable outcomes and defenses against manipulation.

51 Prover-verifier games

Separate solution generation from cheaper verification in an adversarial game.

Watch: Verification is not always easier, especially for open-ended claims.

52 Task decomposition for oversight

Break hard work into smaller units that humans can inspect.

Watch: Local correctness can miss integration failures and hidden dependencies.

53 Consultancy protocol

Let one model advise an interrogating human judge as a baseline to debate.

Watch: The adviser can control framing and omit alternatives.

Interpretability for alignment

Methods 54-65

Investigate internal representations and causal mechanisms, not only outputs. [6][7]

54 Linear probing

Train a simple classifier on activations to test whether a concept is decodable.

Watch: Decodability does not show that the model uses the representation.

55 Sparse autoencoders

Decompose activations into sparse, more interpretable features.

Watch: Features remain incomplete, evaluator-dependent, and sometimes polysemantic.

56 Top-K and gated SAE variants

Improve sparsity control and feature quality in sparse autoencoders.

Watch: Sharper dictionaries can still miss distributed computation.

57 Transcoders

Model transformations between layers to expose computation, not just state.

Watch: Approximations may not faithfully capture the original network.

58 Logit lens and tuned lens

Read intermediate-layer predictions during a forward pass.

Watch: Intermediate readouts can misrepresent nonlinear downstream processing.

59 Activation patching

Swap internal activations across runs to test causal contribution.

Watch: Off-distribution patches and correlated components complicate interpretation.

Interpretability for alignment

Methods 54-65

Investigate internal representations and causal mechanisms, not only outputs. [6][7]

60 Causal scrubbing

Test whether a hypothesized computational graph explains behavior under resampling.

Watch: Results are only as strong as the causal hypothesis and intervention design.

61 Circuit analysis

Reverse-engineer subnetworks that implement a specific computation.

Watch: Detailed success on small circuits may not scale to broad behaviors.

62 Representation engineering

Identify concept directions and intervene on them to change behavior.

Watch: Directions can entangle multiple concepts and vary across contexts.

63 Activation steering

Apply contrastive steering vectors during generation without retraining.

Watch: Effects may be brittle, overbroad, or easy to circumvent.

64 Automated interpretability

Use models to label and explain large numbers of features or neurons.

Watch: Automated explanations require independent fidelity checks.

65 Path patching

Trace specific information-flow paths through network components.

Watch: A selected path may explain one task while missing redundant routes.

Evaluation and red teaming

Methods 66-77

Measure expected behavior and actively search for failure outside the happy path. [8][9]

66 HarmBench

Use a standardized set of harmful behaviors, attacks, and robust-refusal metrics.

Watch: Coverage and evaluator reliability bound every benchmark score.

67 Manual red teaming

Employ skilled humans to discover novel failure modes and attack chains.

Watch: Findings are high-value but hard to reproduce and scale.

68 Automated red teaming

Use attacker models to generate and adapt large test sets.

Watch: Attack-model limitations can create false confidence.

69 Greedy Coordinate Gradient attacks

Optimize adversarial token suffixes using gradients to elicit forbidden behavior.

Watch: White-box success may not transfer; defenses can overfit known suffix patterns.

70 PAIR

Iteratively refine black-box jailbreak prompts using target-model feedback.

Watch: Query cost and evaluator quality affect apparent attack success.

71 Tree of Attacks with Pruning

Search multiple jailbreak branches and discard weak candidates.

Watch: Compute-heavy search can still miss qualitatively different attacks.

Evaluation and red teaming

Methods 66-77

Measure expected behavior and actively search for failure outside the happy path. [8][9]

72 Many-shot jailbreaking

Use many in-context examples to shift the model toward unsafe compliance.

Watch: Long contexts create attack surfaces that short-prompt tests miss.

73 Model-written evaluations

Have models generate tests for latent traits such as sycophancy or power seeking.

Watch: Generated tests reflect the author model's conceptual blind spots.

74 WMDP

Evaluate proxy hazardous knowledge in biology, chemistry, and cybersecurity.

Watch: Knowledge scores are not direct measures of real-world uplift or intent.

75 Situational Awareness Dataset

Test whether a model recognizes facts about itself, evaluation, and deployment.

Watch: Question answering is not the same as strategic situational awareness.

76 Sycophancy evaluations

Measure whether user beliefs distort truth-seeking responses.

Watch: Agreement can be socially appropriate in some contexts; rubrics need nuance.

77 Predeployment safety gates

Tie evaluations to explicit release thresholds and escalation policies.

Watch: Governance works only if thresholds are binding and independently reviewed.

Inference-time techniques

Methods 78-87

Add controls after training, especially when the base model cannot be changed. [2]

78 System prompting

Set persistent roles, constraints, and priorities at the top of context.

Watch: Prompts are soft controls and can conflict with later context.

79 Llama Guard-style classifiers

Use a separate model to classify prompts or outputs against a taxonomy.

Watch: Taxonomy gaps, latency, and false positives remain deployment trade-offs.

80 Constitutional classifiers

Train input-output filters against explicit constitutional rules.

Watch: Classifier robustness and over-refusal need adversarial evaluation.

81 Moderation endpoints

Call specialized services that score content-risk categories.

Watch: Category scores are not a complete policy or action authorization system.

82 Instruction hierarchy

Prioritize system, developer, user, and tool instructions by trusted source.

Watch: Hierarchy helps only when provenance is correctly tracked.

83 Spotlighting for prompt injection

Delimit untrusted content and mark it as data rather than instructions.

Watch: Formatting reduces risk but is not a security boundary.

84 Reasoning-trace monitoring

Inspect available reasoning signals for evidence of unsafe planning.

Watch: Hidden or unfaithful reasoning can evade monitors.

85 Guided or controlled decoding

Alter token probabilities during generation using constraints or discriminators.

Watch: Local token control may harm fluency and miss long-horizon intent.

86 Self-refinement at inference

Generate, critique, and rewrite before returning an answer.

Watch: Extra passes increase latency and can polish a wrong answer.

87 Selective abstention

Decline, defer, or ask for clarification when uncertainty or risk is high.

Watch: Poor calibration can produce either reckless answers or unusable refusal.

Robustness and safety training

Methods 88-95

Harden known attack surfaces and remove targeted capabilities while retaining utility. [9][10]

88 Adversarial jailbreak training

Retrain on successful attacks and safe responses in repeated cycles.

Watch: Adaptive attackers can move beyond the training distribution.

89 Gradient-ascent unlearning

Increase loss on a forget set to suppress targeted knowledge.

Watch: Crude updates can damage neighboring knowledge and be reversible.

90 Gradient-difference unlearning

Forget a target while training normally on a retain set.

Watch: Retain-set coverage determines collateral damage.

91 Negative Preference Optimization

Treat forbidden outputs as rejected responses in a preference objective.

Watch: Suppressed behavior may remain recoverable under prompting or fine-tuning.

92 Representation Misdirection for Unlearning

Perturb hazardous representations while preserving benign activations.

Watch: Benchmark reduction does not establish complete erasure.

93 Task-vector unlearning

Subtract a fine-tuning weight difference associated with target knowledge.

Watch: Knowledge may not be isolated in a clean linear task vector.

94 Circuit breakers

Reroute internal representations away from harmful completion trajectories.

Watch: Robustness across attacks and capability preservation require broad tests.

95 RL with verifiable rewards

Optimize against programmatic checks for math, code, or other verifiable tasks.

Watch: Verifiers cover only measurable aspects and can themselves be gamed.

Multi-agent and societal alignment

Methods 96-101

Address oversight, incentives, interaction, and provenance beyond one model-user pair. [12][13]

96 Corrigibility training

Train systems to accept correction, shutdown, and policy updates.

Watch: Behavioral compliance in tests may fail under new incentives.

97 Mechanism design for AI agents

Structure rules and incentives so self-interested agents produce safer outcomes.

Watch: Mechanisms depend on assumptions about observability and strategic behavior.

98 Cooperative AI and coordination protocols

Build communication, trust, and bargaining methods for interacting agents.

Watch: Cooperation within a group can still harm outsiders.

99 Generative-agent social simulations

Study norms and emergent behavior in simulated multi-agent environments.

Watch: Believable simulation is not predictive validity.

100 Democratic input for model values

Gather structured public input on contested model behaviors.

Watch: Participation, agenda setting, and aggregation determine whose voice counts.

101 AI-output watermarking

Embed detectable signals to support provenance of generated content.

Watch: Watermarks can be removed, degrade with editing, and do not prove truth.

Choose methods by failure model, not by fashion

Deployment condition	Minimum technical stack	Evidence to demand
General assistant, no tools	SFT + preference tuning + principle layer + input/output classifier	Helpfulness, calibration, bias, refusal precision, adaptive jailbreaks
High-stakes domain assistant	Expert demonstrations + abstention + retrieval/tool validation + human escalation	Domain accuracy, uncertainty calibration, subgroup analysis, incident drills
Reasoning model	Verifiable rewards where possible + process checks + outcome tests	Verifier robustness, shortcut tests, hidden-state or trace monitoring limits
Tool-using agent	Instruction hierarchy + least privilege + sandbox + action confirmation + logs	Prompt-injection tests, long-horizon trials, rollback and authorization checks
Multi-agent system	Coordination protocol + mechanism design + independent monitors + circuit breakers	Collusion, correlated failure, escalation, outsider harm, emergent incentives
Frontier training program	Data governance + post-training + scalable oversight + interpretability + release gates	Capability-risk evals, red-team coverage, weight security, external review

Selection rule 1: separate signal from control

A constitution is a specification signal; DPO is an optimization method; a classifier is a runtime control; HarmBench is evidence. Asking one layer to do another layer's job creates category errors.

Selection rule 2: test the interface

Most failures occur between methods: the reward model and policy, the model and tool, the classifier and policy, or the agent and human reviewer. Evaluate compositions, not isolated parts.

Selection rule 3: preserve reversibility

Prefer controls that can be versioned, rolled back, and audited. Keep training datasets, constitutions, evaluators, and thresholds tied to model versions.

Selection rule 4: budget for adversaries

Averages are insufficient for safety. Reserve evaluation capacity for adaptive attacks, distribution shift, and failures that occur only after many steps.

Decision record template: target behavior; affected users; threat model; selected methods; expected benefit; known blind spots; evaluation set; release threshold; owner; rollback trigger.

Turn the map into one small, testable system

The fastest way to learn alignment is to implement a narrow stack, attack it, and explain what each layer failed to guarantee. A small open model or hosted model with a classroom-safe task is enough.

01 Specify and measure

Choose a benign task such as tutoring or code explanation. Write five behavioral principles and 30 test prompts. Label failure types before testing. Deliverable: a versioned rubric and baseline results.

02 Add demonstrations and preferences

Create 50 strong demonstrations and 50 preference pairs. Compare prompt-only behavior with SFT or a lightweight preference method. Deliverable: a table of gains, regressions, and label disagreements.

03 Attack the stack

Run manual and automated red-team prompts, including conflicting instructions and long-context distractions. Add a classifier or abstention rule. Deliverable: attack success rate and false-refusal analysis.

04 Inspect, integrate, and argue

Use one interpretability technique if model access permits; otherwise analyze failure clusters. Produce a safety case that states what evidence supports and what remains unknown.

Suggested team roles

- Specification and dataset lead
- Training and orchestration lead
- Evaluation and statistics lead
- Red-team and security lead

Grading rubric

- 25% reproducibility
- 25% evaluation quality
- 20% failure analysis
- 15% ethical and stakeholder reasoning
- 15% clarity of safety claims

Capstone question: Which failure remains possible after every layer you added, and what evidence would change your mind about the system's safety?

Start with the primary literature

[1] **Ouyang et al. (2022).** Training Language Models to Follow Instructions with Human Feedback.
https://arxiv.org/abs/2203.02155

[2] **Rafailov et al. (2023).** Direct Preference Optimization: Your Language Model is Secretly a Reward Model.
https://arxiv.org/abs/2305.18290

[3] **Bai et al. (2022).** Constitutional AI: Harmlessness from AI Feedback.
https://arxiv.org/abs/2212.08073

[4] **Irving, Christiano, and Amodei (2018).** AI Safety via Debate.
https://arxiv.org/abs/1805.00899v2

[5] **Burns et al. (2023).** Weak-to-Strong Generalization: Eliciting Strong Capabilities with Weak Supervision.
https://arxiv.org/abs/2312.09390

[6] **Bricken et al. (2023).** Towards Monosemanticity: Decomposing Language Models with Dictionary Learning.
https://transformer-circuits.pub/2023/monosemantic-features/

[7] **Templeton et al. (2024).** Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet.
https://transformer-circuits.pub/2024/scaling-monosemanticity/

Reading note: Several entries in the 101-method map are research proposals, wrappers, or close variants rather than settled standards. Read claims as scoped to the cited experiments, model families, datasets, and threat models.

Start with the primary literature

[8] **Mazeika et al. (2024).** HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. http://arxiv.org/pdf/2402.04249v2

[9] **Li et al. (2024).** The WMDP Benchmark: Measuring and Reducing Malicious Use with Unlearning. https://arxiv.org/abs/2403.03218v6

[10] **Zou et al. (2024).** Improving Alignment and Robustness with Circuit Breakers. https://arxiv.org/pdf/2406.04313v2

[11] **Xie et al. (2023).** DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining. https://arxiv.org/abs/2305.10429

[12] **Park et al. (2023).** Generative Agents: Interactive Simulacra of Human Behavior. http://arxiv.org/abs/2304.03442v1

[13] **Dafoe et al. (2020).** Open Problems in Cooperative AI. https://arxiv.org/abs/2012.08630

[14] **Bell et al. (2026).** Beyond Preferences: Learning Alignment Principles Grounded in Human Reasons and Values. https://arxiv.org/abs/2601.18760v1

Reading note: Several entries in the 101-method map are research proposals, wrappers, or close variants rather than settled standards. Read claims as scoped to the cited experiments, model families, datasets, and threat models.

Precise terms prevent category errors

Alignment target	The behavior, objective, constraint, principle set, or outcome a system is meant to satisfy.
Base model	A pretrained model before task-specific instruction or preference tuning.
Distribution shift	A change between training or evaluation conditions and real deployment.
Fine-tuning	Additional parameter updates on a smaller, targeted dataset after pretraining.
Jailbreak	An input strategy intended to bypass a model's safety behavior.
Mechanistic interpretability	Study of internal computations that causally produce model behavior.
Preference pair	Two candidate responses labeled to indicate which is preferred.
Proxy objective	A measurable substitute optimized in place of a richer real-world goal.
Red teaming	Adversarial testing designed to discover failures before or during deployment.
Reward model	A learned function that predicts human or AI judgments of responses.
Scalable oversight	Methods intended to keep evaluation useful when tasks exceed direct human ability.
SFT	Supervised fine-tuning on selected input-output demonstrations.
Unlearning	An intervention intended to reduce a model's access to targeted knowledge or behavior.
Verifier	A program, model, or human process that checks a candidate result.

Scope note: This guide uses “alignment” operationally. It does not assume that language models possess human-like values, intentions, or moral agency; it asks what behaviors and mechanisms appear under defined tests and controls.